

공공데이터 기반 웹 서비스

캠핑나침반

전국 캠핑장을 지역·시설·날씨까지
한 번에 찾는 검색 웹사이트

어디로 캠핑 갈까요? 지역·업종·시설로 검색

검색

Python · Flask · 공공데이터포털 Open API · Open-Meteo · Render 배포

1

프로젝트 소개

왜 만들었고, 무엇을 해결하려 했는가

프로젝트명

캠핑나침반 — 전국 캠핑장 검색 웹사이트

선택한 이유

- ① 인스타·SNS 대신, 이 사이트에서 바로 지역별로 검색·비교
- ② 캠핑장별로 어떤 시설(유틸리티)을 쓸 수 있는지 확인
- ③ 아이 동반·부부·전 연령 누구나 쉽게 이용
- ④ 시간 부족한 사람도 '소요시간'으로 일정 짜기 쉽게

핵심 목적

- ① 캠핑장 위치·운영시설을 지역별로 한눈에 보기
- ② 경유지까지 더해 '가는 시간'을 미리 확인 (시간 부족해도 OK)
- ③ 시설·유틸리티를 비교해 '필요한 시설' 위주로 예약 선택

2

수집한 데이터

어디서, 어떻게 모았는가 (공공데이터포털만 사용)

어떤 데이터

전국 야영장(캠핑장)의 상세 정보

데이터 출처

공공데이터포털 (data.go.kr) 에서만 수집
— 한국관광공사 '고캠핑' Open API 제공

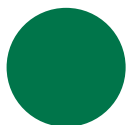
수집 방식

웹 크롤링 X → 공식 Open API 활용 (JSON으로 수신)

수집한 주요 필드

- 캠핑장명
- 주소 · 위도/경도
- 업종 (일반·글램핑·카라반...)
- 주요시설 (전기·온수·화장실...)
- 데크·파쇄석·카라반 사이트 수
- 대표사진 · 예약 페이지 URL

약 3,800곳 의 캠핑장 데이터



데이터 · API 출처

이 서비스가 쓴 데이터·API와 어디서 가져왔는지 (모두 무료)

캠핑장 데이터

공공데이터포털 (data.go.kr)

한국관광공사 '고캠핑' Open API

무료 · 인증키 발급

소요시간 · 경로

네이버 클라우드 플랫폼

Maps · Directions(길찾기) API

이용 신청 · Key ID / Secret · 300건까지 무료

장소 검색 (경유지)

카카오 디벨로퍼스

로컬(Local) 키워드 검색 API

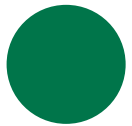
무료 · REST API 키

주말 날씨

Open-Meteo (open-meteo.com)

Weather Forecast API

무료 · 키 불필요



API 활용 vs 웹 크롤링

이 프로젝트가 데이터를 가져온 방식

웹 크롤링

- 사람이 보는 웹페이지(HTML)를 통째로 긁어와 태그를 분석해 원하는 값을 뽑아냄
- 도구: requests + BeautifulSoup
- 화면 구조가 바뀌면 깨질 수 있음

API 활용

← 이 프로젝트 방식

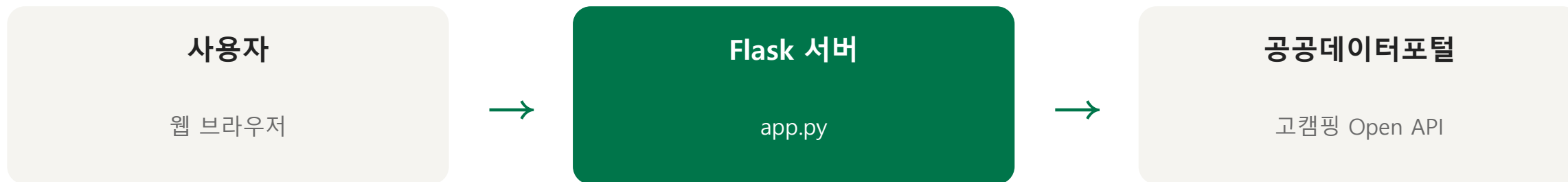
- 제공자가 공식으로 연 '데이터 창구(API)'에 요청 → 정리된 JSON을 바로 받음
- 도구: requests + json (HTML 파싱 없음)
- 안정적이고 정식 통로

이 프로젝트는 100% API 활용입니다. 공공데이터포털의 '고캠핑' Open API에 공식 요청을 보내 정리된 JSON 데이터를 받아 사용했습니다. (웹페이지 HTML을 긁는 '크롤링'은 쓰지 않았습니다)

3

시스템 구성

사용자 요청을 받아 공공데이터를 화면으로 바꿔주는 구조



+ 부가 연동 · 날씨: Open-Meteo API · 소요시간: 네이버 지도 길찾기 · 배포: Render + gunicorn

사용한 기술

Python · Flask · requests · Jinja2(HTML 템플릿) · HTML / CSS / JavaScript · gunicorn · Render

3

프로그램 동작 과정

검색 한 번에 일어나는 3단계

1

사용자가 검색

지역(시도·시군구)을 고르거나
'글램핑','전기' 같은 키워드 입력



2

서버가 골라냄

전국 데이터 중 조건에 맞는
캠핑장만 걸러낸다 (필터링)



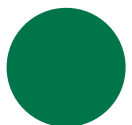
3

화면에 표시

사진·시설·사이트 수 카드로 보여주고
날씨·거리도 함께 표시

+ 데이터 수집 과정 (맨 처음 한 번) — 크롤링이 아니라 API 호출입니다

공공데이터포털의 Open API는 데이터를 한 번에 다 주지 않고 '페이지'로 나눠 줍니다. 그래서 페이지를
끝까지 넘기며 전국 캠핑장을 모두 받아 서버 메모리에 저장(캐시)해 둡니다.



핵심 코드 ① 데이터 수집 (공공데이터 API 활용)

공공데이터포털 Open API를 페이지 단위로 끝까지 받아 모으기

```
def fetch_all_items():
    all_items = []
    page = 1
    while True:
        # 다음 페이지를 순차 호출
        res = requests.get(
            BASE_URL,
            params={"pageNo": page, "numOfRows": 100}
        )
        items = res.json()["response"]["body"]["items"]["item"]
        if not items:
            break # 데이터 없음 확인 시 루프 탈출
        all_items += items
        page += 1
    return all_items
```

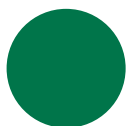
쉽게 말하면

공공데이터포털 API는 한 번에 다 주지 않고 여러 '페이지'로 나눠 줌

그래서 페이지를 1, 2, 3... 끝까지 넘기며 전국 캠핑장을 전부 모아 서버에 저장해 둡니다.
(HTML을 긁는 게 아니라 API 호출)

POINT

데이터를 미리 다 받아 서버 메모리에 저장 → 두 번째 검색부터 훨씬 빨라집니다.



핵심 코드 ② 검색 · 필터

영어 필드를 한글로 바꾸고, 조건에 맞는 곳만 고르기

```
FIELD_MAP = {
    "facItnm": "캠핑장명", "induty": "업종",
    "addr1": "주소", "sbrsCl": "주요시설"
}

for row in campsites:
    # 비교 검색용 문자 가공 (공백 제거)
    haystack = (row["facItnm"]+row["addr1"]+row["sbrsCl"]).replace(" ", "")
    clean_keyword = keyword.replace(" ", "")

    if clean_keyword in haystack:
        result.append(row)
```

쉽게 말하면

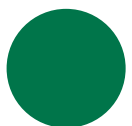
API가 주는 이름은 영어라서
(facItnm) 한글로 바꿔줍니다.

그리고 사용자가 넣은 지역·키워드
(글램핑, 전기...)가 들어있는
캠핑장만 골라냅니다.

'금호강오토캠핑장'처럼 붙여 써도
찾히도록 띄어쓰기는 무시합니다.

POINT

검색·필터 로직은 이런 사이트의 '심장'입니다. 무엇으로 어떻게 걸러줄지가 서비스의 핵심.



핵심 코드 ③ Flask 라우트 + 화면 연결

요청을 받아 데이터를 골라 화면으로 돌려주기

```
@app.route("/")
def index():
    # 사용자가 브라우저에서 보낸 파라미터 획득
    region = request.args.get("region", "")
    sido = request.args.get("sido", "")
    sigungu = request.args.get("sigungu", "")

    # 필터 필터링 메서드 실행
    rows = get_campsites(region, sido, sigungu)

    # Jinja2 템플릿 파일로 완성된 데이터 바인딩
    return render_template("index.html", rows=rows)
```

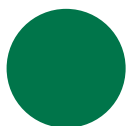
쉽게 말하면

@app.route 는 '주소'와 '함수'를
연결해 줍니다.

누가 우리 사이트에 들어오면
→ 조건에 맞는 캠핑장을 고르고
→ HTML 화면에 담아
→ 브라우저로 돌려줍니다.

POINT

'요청 → 데이터 → 화면 반환'. 이 흐름이 모든 웹서버(백엔드)의 기본기입니다.



핵심 코드 ④ 외부 API 연동 — 소요시간 & 경유지

네이버 길찾기로 '현위치 →(마트)→ 캠핑장' 시간 계산 · 카카오로 마트 찾기

```
// (프론트) 사용자 모바일 GPS 위경도 확보
navigator.geolocation.getCurrentPosition((pos) => {
  let lat = pos.coords.latitude;
  let lng = pos.coords.longitude;
  let url = `/api/duration?slat=${lat}&glng=${lng}...`

  // 경유지가 있으면 쿼리스트링에 추가 전송
  if (waypoint) {
    url += `&wlat=${waypoint.lat}&wlng=${waypoint.lng}`
  }
  fetch(url).then(r => r.json()).then(showTime);
});
```

쉽게 말하면

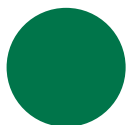
·기본: 내 위치 → 캠핑장까지
네이버 길찾기로 소요시간 계산
(캠핑장 좌표는 공공데이터 API 제공)

·경유지: 카카오로 '내 주변 마트'를
찾아 그 좌표를 요청에 이어붙이면
'현위치 → 마트 → 캠핑장'으로 계산

화면 전체 새로고침 없이! (=AJAX)

POINT

네이버(길찾기)로 시간 계산 + 카카오(로컬 검색)로 경유지 찾기 — 두 API를 이어 실제 동선대로 계산해요.



핵심 코드 ⑤ 배포 (Render)

'내 컴퓨터'를 넘어 누구나 쓰는 사이트로 — 타임아웃까지 맞추기

```
# [Render 서비스용 실행 명령어 세팅]
gunicorn app:app --workers 1 --timeout 120

/* [모바일 폰 환경 반응형 매체 전환] */
@media (max-width: 680px) {
  .camp-card { flex-direction: column; }
  .thumbnail { width: 100%; height: 200px; }
}
```

쉽게 말하면

내 컴퓨터에서만 되던 걸 인터넷에 공개하려면 '배포'가 필요합니다.

개발용 서버 대신 gunicorn을 쓰고 Render에 올렸고

타임아웃(--timeout 120)은 두 곳에 똑같이 넣었습니다:

- ① 실행 명령(코드)
- ② Render 대시보드 Start Command

그리고 모바일 적용이 가능케 하기위해 680px로 검색결과가 잘보일수있게 설정했음.

POINT

첫 검색은 전국 데이터를 한꺼번에 받아 30초를 넘길 수 있어요. 기본 타임아웃이면 끊기니 120초로 늘렸습니다.

4

구현 화면

메인 · 검색 결과 · CSV 다운로드

대구광역시

군위군

Q 검색

장보기 경유지

검색

이마트24 대구감삼대박점 X

20개 검색됨 — 대구광역시 군위군

↓ 엑셀(CSV)로 저장



80 SS 오토캠핑장

자동차야영장

◆ 대구광역시 군위군 부계면 한티로 1525-6

☎ 054-382-0007

운영 민간

관리 직영

데크 24

시설

전기

무선인터넷

장작판매

온수

물놀이장

놀이터

운동시설

화장실 2개

샤워실 2개

소요시간

예약

📍 현재위치

→

🛒 이마트24 대구감삼대박점

→

🏕️ 캠핑장

약 37분 · 31.6km

🛒 이마트24 대구감삼대박점 경유

주말 날씨 (금·토·일)

금

07/24



30°/24°

토

07/25



25°/23°

일

07/26



31°/23°

검색 결과 — 사진·시설·소요시간(경유지)·주말 날씨, CSV(엑셀) 다운로드

5

어려웠던 점 & 해결

문제를 만나고, 하나씩 해결한 과정

!

검색 결과 수가 고캠핑 사이트와 달랐다

→

Open API 등록 수와 고캠핑 사이트 등록 수가 달라서 생긴 '구조적 한계' (검색 로직은 업종·띄어쓰기로 보완)

!

Open API에 사진·정보가 빠진 곳이 많았다

→

있는 데이터만 표시(사진 없으면 기본 이미지), 추후 지속 업데이트로 보완 예정

!

배포했더니 사이트가 안 떴다

→

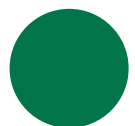
개발용 서버 대신 gunicorn 사용 + 포트·타임아웃(120초) 설정

!

날씨가 '한도 초과'로 안 나왔다

→

좌표 묶기 + 캐시 + 주말 3일치만 요청 (다음 장에서 자세히)



트러블슈팅 — 날씨 API '한도 초과'

왜 생겼고, 어떻게 해결했나

왜 생겼나 (원인)

- 무료 날씨 서비스(Open-Meteo)는 하루에 부를 수 있는 횟수 (무료 한도)가 정해져 있습니다.
- 처음엔 검색할 때마다 캠핑장 20곳 날씨를 매번 새로 요청했어요.
- 테스트를 반복하니 하루 한도를 금방 넘겨 버렸습니다.
→ "Daily request limit exceeded"

어떻게 해결했나

- ① 좌표 반올림 — 가까운 캠핑장은 같은 좌표로 묶어 요청 수를 줄임
- ② 캐시(임시 저장) — 한 번 받은 날씨는 3시간 동안 재사용 (같은 곳 반복 요청 X)
- ③ 딱 필요한 만큼 — 10일치 대신 주말 3일치만 요청

POINT

핵심은 '덜 자주 · 덜 많이 · 받은 건 재사용'. 호출량을 확 줄여 한도 문제를 없앴습니다.

6

느낀 점 & 개선 계획

배운 것, 그리고 앞으로 더하고 싶은 것

배운 점

- 이제까지 배운 함수·메서드를 최대한 활용해 구현
- 공공데이터 API를 실제 서비스로 만들어봤다
- API 연동(소요시간·날씨)을 직접 해봤다
- 내 손으로 인터넷에 배포까지 해봤다
- 오류를 로그로 찾아 고치는 과정을 익혔다

개선하고 싶은 기능

- 실시간 예약 현황 · 현재 위치 기반 '빈자리' 추천
- 통합 예약 — 내 사이트에서 바로 예약·결제 연동
- 요금 비교 + '저렴한 공공 캠핑장 순' 정렬
- 후기·평점 · 날씨·거리 기반 AI 맞춤 추천
- 공공데이터 자동 갱신 + 크롤링으로 누락 보완

감사합니다

공공데이터로 만든 전국 캠핑장 검색 서비스,
캠핑나침반.

camping-scrapper.onrender.com

Q & A